

C O D E Q U A L I T Y R E V I E W

Software Engineering Quality Analysis

Automated Examination Behavior Proctoring System

Project Repository: github.com/Paryamiel/prototype

Evidence Base: Source code, tests, artefacts and git history

Prepared: September 2026

Table of Contents

Executive Summary 1

Project Overview and Repository Distribution.....2

 The Software System2

 Repository Distribution and Cloning3

Instructions for Running the Project on Another Device3

 Prerequisites3

 Installation and Environment Verification.....3

 Running the System4

 Known Operational Notes.....5

Evaluation Scope, Evidence Base and Method.....6

Finding 1: Professional Coding Standards and Practices6

 Implemented Standards and Practices6

 Standards Not Implemented (Gaps).....7

Finding 2: The Fifteen Principles of Quality Software.....7

 Principle-by-Principle Assessment7

 Discussion of Key Evidence8

 Principles Only Partially Satisfied8

Finding 3: Defensive Programming8

 Implemented Defensive Techniques.....8

 Missing Defensive Measures (Gaps)10

Finding 4: Error and Exception Handling11

 Implemented Exception Handling11

 Missing Exception Handling (Gaps).....11

Consolidated Findings and Recommendations12

Conclusion and Overall Verdict12

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

Executive Summary

This report documents a structured quality assessment of the Automated Examination Behavior Proctoring System, a software engineering group project hosted at github.com/Paryamiel/prototype. The system classifies examination conduct from video files or a live camera into five behaviour classes – normal, hand_signal, passing_of_notes, side_glancing, and use_of_unauthorized_object – by combining YOLO-based human pose estimation, ByteTrack multi-object tracking, and a bidirectional Long Short-Term Memory (LSTM) classifier implemented in PyTorch. The assessment was prepared in connection with the upload of the project root folder to the designated group GitHub repository, together with run instructions that allow every group member to execute the project on their own device.

Every finding in this report is grounded in the actual source code, and each one cites the responsible file together with line numbers. Four prescribed categories were evaluated: first, professional coding standards and practices; second, fifteen principles of quality software (correctness, reliability, efficiency, usability, maintainability, flexibility, portability, reusability, testability, security, modularity, readability, robustness, simplicity, and consistency); third, defensive programming; and fourth, error and exception handling. For each category, the report identifies the code that implements the principle as well as the code that fails to implement it.

The overall result is positive. **Twelve of the fifteen quality principles are fully satisfied**, and the remaining three (portability, security, and consistency) are only partially satisfied. Professional standards such as PEP 8 naming, type annotations, Google-style docstrings, centralised configuration, and conventional commits are applied consistently throughout the codebase. Defensive techniques concentrate at the perception boundary, where keypoints, tracking results, and dataset shapes are validated through multi-level fallbacks. The most significant weaknesses are concentrated in three areas: assert-based runtime validation that Python removes under optimisation, the absence of input-range validation for command-line arguments, and missing exception guards around model-loading and training code paths. Table 1 summarises the outcome per category.

Table 1: Summary matrix of implemented items versus identified gaps per evaluation category

Evaluation Category	Implemented Items	Identified Gaps	Overall Assessment
Professional Coding Standards and Practices	11 practices verified	6 gaps	Strong foundation; tooling and documentation gaps
15 Principles of Quality Software	12 fully satisfied	3 partially satisfied	Solid architectural quality
Defensive Programming	9 techniques verified	6 gaps	Strong at data boundaries; weaker at CLI boundaries
Error and Exception Handling	5 mechanisms verified	5 gaps	Sound in the real-time path; absent in batch paths

Project Overview and Repository Distribution

The Software System

The repository implements an end-to-end pipeline in which every video frame is first processed by a YOLO pose-estimation model that extracts seventeen COCO skeletal keypoints for each detected student. ByteTrack then assigns persistent track identifiers so that temporal sequences always belong to the same person. Keypoints are normalised relative to the torso midpoint and bounding-box scale (`dataset.py`, lines 17–77), which makes the features invariant to camera distance, resolution, and seat position. For every tracked student, a per-person ring buffer of thirty consecutive frames (approximately one second at 30 fps) feeds a bidirectional LSTM whose temporal representations are classified into the five behaviour classes by a multilayer-perceptron head. A real-time proctoring interface renders bounding boxes, colour-coded skeletons, confidence badges, and debounced alert banners.

The project root also ships with reproducible artefacts: a processed demonstration dataset (`data/processed/exam_dataset.npz`, 500 samples of shape 30×34), a trained checkpoint (`weights/best_lstm_model.pt`), and evaluation reports under `reports/`. It should be noted for academic honesty that the metadata embedded in the dataset file records its source as a synthetic demonstration generator, and the reported test accuracy of 100 per cent (with fifteen test samples per class) reflects the easy separability of that synthetic data rather than measured real-world performance. Table 2 maps each repository component to its responsibility.

Table 2: Repository components and their responsibilities

Component	Responsibility
<code>config.py</code>	Central configuration: class list, paths, YOLO weight resolution, and all hyperparameters
<code>model.py</code>	ExamBehaviorLSTM architecture: input LayerNorm, two-layer BiLSTM, temporal pooling, five-class head
<code>dataset.py</code>	Keypoint normalisation, PyTorch Dataset wrapper, stratified 70-15-15 split, dataset persistence
<code>tracker.py</code>	PoseTrackerManager: YOLO pose plus ByteTrack tracking and per-student ring buffers
<code>extract_dataset.py</code>	Sliding-window feature extraction from labelled clips; seeded synthetic data generator
<code>train.py</code>	Training loop: weighted loss, AdamW, LR scheduling, early stopping, metric reporting
<code>inference.py</code>	Real-time proctoring engine: HUD, overlays, alert debouncing, annotated video export
<code>clipper.py</code>	Keyboard-driven clip labelling GUI, adapted from IntegraPose under AGPL-3.0
<code>test_prototype.py</code>	Automated verification suite covering four end-to-end checks
<code>data/</code> , <code>weights/</code> , <code>reports/</code>	Shipped artefacts: processed dataset, trained checkpoint, evaluation reports

Repository Distribution and Cloning

The main (root) folder of the project is uploaded to the designated group GitHub repository so that all group members share one canonical copy of the source code, the documentation, and the

artefacts. A `.gitignore` file is committed at the repository root; it excludes byte-compiled files, virtual environments, raw video formats (`*.mp4`, `*.avi`, `*.mov`, `*.mkv`, `*.webm`), large model formats (`*.engine`, `*.onnx`), and temporary test artefacts, which keeps the repository lightweight enough for ordinary cloning. Because large media files are excluded, group members should record or copy their own raw examination footage into `data/raw_videos/<class>/` locally rather than expecting it from the repository.

Any group member can obtain and verify the project with two commands. The first command, `git clone https://github.com/Paryamiel/prototype.git`, downloads the full project folder; the second, `python test_prototype.py` (run after installation as described in the next section), confirms that the environment is capable of executing the pipeline. The repository README additionally documents the step-by-step workflow from clip labelling to live inference, so the repository is self-explanatory for new collaborators.

Instructions for Running the Project on Another Device

Prerequisites

The project targets Python 3.10 to 3.13; the README documents the development environment as Windows 11 with Python 3.13, but the code itself is operating-system agnostic because all paths are constructed with `pathlib` (`config.py`, lines 5–9). An NVIDIA GPU with CUDA accelerates training and inference, although it is optional: both `train.py` (line 159) and `inference.py` (line 163) automatically fall back to the CPU through `torch.device("cuda" if torch.cuda.is_available() else "cpu")`. Approximately 5 GB of free disk space should be reserved for the PyTorch installation, and Git is required to clone the repository.

Installation and Environment Verification

After cloning, every group member should perform the following steps in order:

1. `git clone https://github.com/Paryamiel/prototype.git` followed by `cd prototype` to enter the project root.
2. Create an isolated virtual environment: on Windows PowerShell, `python -m venv venv` then `venv\Scripts\Activate.ps1`; on macOS or Linux, `python3 -m venv venv` then `source venv/bin/activate`.
3. Install the pinned dependencies with `pip install -r requirements.txt`. The file specifies minimum versions for ultralytics, torch, torchvision, opencv-python, numpy, matplotlib, lap, pyyaml, and pillow (`requirements.txt`, lines 1–9).
4. Verify GPU availability with `python -c "import torch; print('CUDA:', torch.cuda.is_available())"`. A printed False simply means training will run on the CPU, which is slower but fully supported.

Running the System

Two paths exist. The demonstration path requires no recorded footage and validates the complete pipeline in minutes, while the full path processes real examination recordings. The demonstration path proceeds as follows:

1. Generate a synthetic five-class dataset: `python extract_dataset.py --generate-synthetic`. This writes 500 sequences to `data/processed/exam_dataset.npz`.

2. Train the classifier: `python train.py --epochs 30 --batch-size 32`. The script performs the stratified 70-15-15 split, applies early stopping, and stores the best checkpoint in `weights/best_lstm_model.pt`.
3. Start proctoring: `python inference.py --source 0` for the live webcam, or `python inference.py --source path/to/exam.mp4` for a video file. Append `--save-video out.mp4` to record the annotated stream, or `--no-display` for headless execution.

The full path replaces the synthetic step with labelled real footage:

1. Launch the labelling GUI with `python clipper.py`, load a raw recording, and cut behaviour segments using the keys n (normal), h (hand_signal), p (passing_of_notes), s (side_glancing), and u (use_of_unauthorized_object). Exported clips land directly in `data/raw_videos/<class>/`.
2. Extract pose sequences with `python extract_dataset.py`, which tracks every person, normalises keypoints, and slices thirty-frame windows with stride five.
3. Retrain with `python train.py --epochs 30 --batch-size 32`. Inspection of `reports/confusion_matrix.png` and `reports/test_evaluation_report.txt` then reflects real footage instead of synthetic data.
4. Run inference as described above, optionally raising or lowering the alert threshold with `--threshold` (default 0.60).

Finally, the automated verification suite should be executed after installation: `python test_prototype.py`. The expected output is four consecutive PASS lines covering keypoint normalisation, the model forward pass, the stratified split, and an end-to-end synthetic training cycle. If all four pass, the installation is correct.

Known Operational Notes

Three practical caveats apply when running the project on different devices, and all of them originate from the project documentation. First, OpenCV on Windows fails silently on non-ASCII file paths, so recordings should be placed in ASCII-only directories. Second, some H.265/HEVC recordings are not decodable by OpenCV and should be re-encoded to H.264 first, for example with `ffmpeg -i input.mp4 -c:v libx264 -crf 18 output.mp4`. Third, the clipper GUI requires the pillow package (listed in requirements.txt) and responds to keyboard shortcuts only when the video canvas has input focus. These notes are reproduced from INTEGRATION.md, lines 183–211, so that group members do not rediscover them by trial and error.

Evaluation Scope, Evidence Base and Method

The assessment is based on a complete source-code inspection of the nine Python modules (approximately 2,150 lines in total), the central configuration file, the dependency manifest, the committed documentation (README.md and INTEGRATION.md), the shipped artefacts, and the git history of the repository. The git history contains three commits – 57458b9 (initial prototype), 49de3ff (removal of a retired two-class scheme), and 136dd20 (clipper integration) – and all line numbers cited in this report refer to the repository state at commit 136dd20, the most recent commit at the time of analysis.

Each of the four prescribed categories was operationalised into concrete, verifiable code-level indicators before the inspection began. Professional standards were checked against naming

conventions, annotation coverage, documentation coverage, configuration management, dependency management, and version-control practice. The fifteen principles of quality software were assessed against the widely taught set comprising correctness, reliability, efficiency, usability, maintainability, flexibility, portability, reusability, testability, security, modularity, readability, robustness, simplicity, and consistency. Defensive programming was assessed by locating every guard, fallback, clamp, and validation branch, and exception handling was assessed by locating every try, except, finally, and deliberate raise. A finding is reported as a gap only when a concrete code location can be cited for its absence, which keeps the assessment reproducible. The scope is limited to static analysis; no runtime fault injection or profiling was performed.

Finding 1: Professional Coding Standards and Practices

Implemented Standards and Practices

Eleven distinct professional practices are verifiably implemented. Naming follows PEP 8 throughout: classes use PascalCase (`ExamBehaviorLSTM` in `model.py` line 15, `PoseTrackerManager` in `tracker.py` line 46, `VideoClipperApp` in `clipper.py` line 78), functions and variables use snake_case, and module-level constants use UPPER_SNAKE_CASE (`config.py`, lines 57–90). Nearly every function signature carries type annotations and a Google-style docstring with Args and Returns sections, as the forward method of the classifier demonstrates:

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Forward pass.
    Args:
        x (torch.Tensor): Input sequence tensor of shape (batch_size, seq_len, input_dim)
    Returns:
        torch.Tensor: Raw logits of shape (batch_size, num_classes)
    """
```

Source: model.py, lines 68–75.

All tunable values live in a single configuration module, so no magic numbers are scattered across the pipeline; the inference engine, for example, imports the alert threshold and debounce window from `config.py` instead of hard-coding them (`inference.py`, lines 11–19). Every executable exposes a documented argparse command-line interface (`train.py` lines 380–398, `extract_dataset.py` lines 277–293, `inference.py` lines 287–305, `clipper.py` lines 457–465), dependencies are version-floored in `requirements.txt`, and all executable modules use the `if __name__ == "__main__":` entry-point guard. The git history follows conventional-commit style (“feat: integrate video clipper GUI, config, and documentation”, commit 136dd20; “chore: remove suspicious folder and aliases...”, commit 49de3ff). The clipper module additionally carries a full attribution header (`clipper.py`, lines 1–27) documenting its IntegraPose origin, upstream commit, DOI, and AGPL-3.0 licence – disciplined intellectual-property practice. Finally, the project includes a dedicated verification suite rather than relying on manual testing:

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger(__name__)
```


Source: clipper.py, lines 70–75 – the only module that configures structured logging (see gaps below).

Standards Not Implemented (Gaps)

Six gaps are identified. First, no automated style enforcement is committed: the repository contains no linter or formatter configuration (no `.flake8`, `pyproject.toml`, or `.editorconfig`), so conformance to the observed conventions relies on developer discipline rather than tooling. Second, the codebase is inconsistent in its observability mechanism: `clipper.py` configures the standard `logging` module properly, while the other eight modules report through bare `print()` calls, which cannot be filtered, timestamped, or redirected uniformly. Third, two docstrings have drifted out of date and now misdescribe the system: `model.py` lines 18–19 still say the model distinguishes “normal’ vs ‘suspicious’ classes” and `dataset.py` line 87 says labels are “(0 or 1)”, both remnants of the retired two-class design removed in commit 49de3ff; the system now classifies five classes. Fourth, `inference.py` lines 44–50 duplicate fallback constants (`input_dim=34`, `num_classes=2`) instead of importing them from `config.py`, and the `num_classes` default of 2 is actively wrong for the five-class system. Fifth, there is no continuous-integration workflow and no contributing guide, so the verification suite in `test_prototype.py` runs only when a developer remembers to invoke it. Sixth, occasional lines exceed common length conventions (for example `dataset.py` lines 63–66), which a configured formatter would normalise.

Finding 2: The Fifteen Principles of Quality Software

Principle-by-Principle Assessment

Table 3 assesses the codebase against the fifteen principles and cites the strongest single piece of evidence for each verdict. Twelve principles are fully satisfied and three – portability, security, and consistency – are only partially satisfied, each for a specific, locatable reason.

Table 3: Assessment of the fifteen principles of quality software

Principle	Status	Primary Evidence (file, lines)
Correctness	Implemented	Stratified per-class 70-15-15 partitioning with ratio assertion – <code>dataset.py</code> 100–166; honest held-out test evaluation – <code>train.py</code> 311–355
Reliability	Implemented	Early stopping with best-checkpoint restore – <code>train.py</code> 287–319; alert debounce – <code>inference.py</code> 232–241
Efficiency	Implemented	O(1) deque ring buffers – <code>tracker.py</code> 22–23; stride-5 windowing that cuts extraction redundancy roughly fivefold – <code>config.py</code> 67, <code>extract_dataset.py</code> 75–82
Usability	Implemented	Live HUD with FPS, student count and alert statistics – <code>inference.py</code> 133–152; actionable next-step messages – <code>train.py</code> 169–173
Maintainability	Implemented	Single-responsibility modules wired through one central configuration file – <code>config.py</code>
Flexibility	Implemented	Every key parameter overridable via <code>argparse</code> – <code>train.py</code> 380–398, <code>inference.py</code> 287–305; device-agnostic execution – <code>train.py</code> 159
Portability	Partial	<code>pathlib</code> -based paths and CPU fallback – <code>config.py</code> 5–9, <code>train.py</code> 159; but README workflow is Windows-specific and <code>config.py</code> 19–20 probes a weight file outside the repository
Reusability	Implemented	Pure function <code>normalize_keypoints</code> reused by the tracker – <code>dataset.py</code> 17–77 imported at <code>tracker.py</code> 13 and called at 125

Principle	Status	Primary Evidence (file, lines)
Testability	Implemented	Four-test verification suite – test_prototype.py 30–110; seeded synthetic generator (RandomState(42)) – extract_dataset.py 95
Security	Partial	No eval/exec usage anywhere; but np.load(allow_pickle=True) – dataset.py 188 – can execute arbitrary code if a malicious .npz is loaded
Modularity	Implemented	Layered architecture config → model/dataset → tracker → extract/train → inference; GUI decoupled from pipeline via data folders
Readability	Implemented	Descriptive identifiers (is_sequence_ready, _prune_stale_tracks) and sectioned GUI construction – clipper.py 124–210, 222–433
Robustness	Implemented	Multi-level keypoint fallbacks – dataset.py 34–69; None-guards on every tracker output – tracker.py 96–111
Simplicity	Implemented	Compact BiLSTM with last-step plus mean pooling – model.py 82–89; metrics computed in plain numpy, avoiding an extra dependency – train.py 39–82
Consistency	Partial	Uniform class indexing via CLASS_TO_IDX/IDX_TO_CLASS – config.py 46–47; but print-based reporting in eight modules versus logging in clipper.py

Discussion of Key Evidence

Three principles deserve brief elaboration because their implementation is unusually careful for a prototype. Correctness is protected structurally rather than by convention: the split function partitions each class separately, guarantees at least one validation and one test sample whenever a class has three or more examples (dataset.py, lines 141–143), and asserts that the ratios sum to one, which prevents silent distribution skew. Robustness rests on a fallback chain with numeric floors – the normalisation function degrades from mid-hip to mid-shoulder to bounding-box centre to the mean of valid points, masks zero coordinates, and clamps the scale factor to a minimum of 0.001 – so that a partially occluded skeleton produces a defined, finite feature vector instead of NaN values. Simplicity is evident in the deliberate avoidance of unnecessary machinery: the temporal representation is a plain concatenation of the last LSTM step and the mean over all thirty steps (model.py, lines 82–89), and the evaluation metrics are computed from a hand-rolled confusion matrix in numpy instead of adding scikit-learn to the dependency tree (train.py, lines 39–82).

Principles Only Partially Satisfied

Portability is weakened by two machine-specific assumptions. The README documents the workflow in Windows PowerShell terms, and the YOLO weight resolver probes `BASE_DIR.parent / "yolo25s-pose"` (config.py, lines 19–20), a sibling directory that exists on the original developer’s machine but not after a clean clone; the ultimate fallback to `yolo11s-pose.pt` (line 33) rescues the run, but only after a silent deviation from the documented setup. Security is mostly sound – there is no dynamic code evaluation and paths are constructed with pathlib – but `np.load(..., allow_pickle=True)` (dataset.py, line 188) permits arbitrary code execution when loading a tampered dataset file; the risk is low in a trusted coursework setting yet should be recorded and eventually removed. Consistency suffers from the print-versus-logging divergence already described under Finding 1, which means the fifteen-principle verdict for consistency mirrors the standards verdict for observability.

Finding 3: Defensive Programming

Implemented Defensive Techniques

Nine defensive techniques are verifiably implemented, and they cluster exactly where the data is least trustworthy. The most sophisticated is the fallback chain in keypoint normalisation, which refuses to trust any single anatomical reference:

```
l_hip, r_hip = coords[11], coords[12]
l_shoulder, r_shoulder = coords[5], coords[6]

# Check if hips are non-zero
if np.any(l_hip > 0) and np.any(r_hip > 0):
    center_x = (l_hip[0] + r_hip[0]) / 2.0
    center_y = (l_hip[1] + r_hip[1]) / 2.0
elif np.any(l_shoulder > 0) and np.any(r_shoulder > 0):
    center_x = (l_shoulder[0] + r_shoulder[0]) / 2.0
```

Source: dataset.py, lines 35–43; the chain continues through a bounding-box centre (lines 45–47) to a mean-of-valid-points fallback (lines 48–54).

The tracker treats every model output as untrusted until proven otherwise. Empty result sets, missing boxes or keypoints, and unassigned track identifiers are all intercepted before any downstream code dereferences them:

```
if boxes is None or len(boxes) == 0 or keypoints is None or len(keypoints) == 0:
    self._prune_stale_tracks()
    return frame_detections

# Check track IDs
if boxes.id is None:
    # Detections without assigned track IDs yet
    return frame_detections
```

Source: tracker.py, lines 104–111.

Configuration resolution is defensive in the same spirit: the loader probes six candidate weight locations and degrades gracefully through an engine file, a repository-root file, and finally the ultralytics built-in name, so a missing custom weight never crashes the program at import time:

```
def resolve_yolo_weights() -> str:
    """Find the best available YOLO pose weight file."""
    for p in _CANDIDATE_YOLO_WEIGHTS:
        if p.exists():
            return str(p)
    default_path = BASE_DIR.parent / "yolo25s-pose" / "yolo26s-pose.pt"
    if default_path.exists():
        return str(default_path)
    return "yolo11s-pose.pt"
```

Source: config.py, lines 25–33.

The alerting path applies hysteresis so that a single noisy frame cannot flip a student into an alert state or leave one stuck there: the counter rises by two only when a suspicious class exceeds the confidence threshold, is capped at twice the debounce window, and decays by one toward zero otherwise,

```

if pred_idx != 0 and conf >= threshold:
    person.alert_counter = min(ALERT_DEBOUNCE_FRAMES * 2, person.alert_counter + 2)
    current_frame_suspicious += 1
    person.current_prediction = predicted_class
    person.current_confidence = conf
else:
    person.alert_counter = max(0, person.alert_counter - 1)
    if person.alert_counter == 0:
        person.current_prediction = "normal"

```

Source: inference.py, lines 232–240.

The remaining four techniques are: numeric floors that prevent division by zero and degenerate scales (`np.maximum(class_counts, 1)` in `train.py` line 204; `max(1, int(1000 / self.fps))` in `clipper.py` line 317; `scale = max(scale, 1e-3)` in `dataset.py` line 69); strict sequence-shape validation before any window enters the training set (`extract_dataset.py`, line 79); explicit preconditions and range validation in the GUI, which rejects incomplete clip selections and inverted frame ranges with explanatory dialogs (`clipper.py`, lines 358–363); and clamped seeking, where the target frame is constrained to `max(0, min(int(frame_num), total_frames - 1))` (`clipper.py`, line 279) and single-frame stepping is bounds-checked (line 327). Skeleton rendering additionally draws joints only at strictly positive coordinates (`inference.py`, lines 111 and 121), so absent keypoints can never be plotted at the origin.

Missing Defensive Measures (Gaps)

Six gaps are identified, all at the command-line and persistence boundaries. First, runtime validation is delegated to `assert` statements, which Python silently removes under the `-O` optimisation flag:

```

# dataset.py, line 89
assert len(X) == len(y), f"Mismatched data sizes: X={len(X)}, y={len(y)}"

# dataset.py, line 123
assert np.isclose(train_ratio + val_ratio + test_ratio, 1.0), "Splits must sum to 1.0"

```

Source: dataset.py, lines 89 and 123 – asserts are debugging aids, not input validation.

Second, none of the four argparse interfaces validates ranges: `--threshold` accepts values outside the unit interval (`inference.py`, line 294), and `--epochs`, `--stride`, and `--batch-size` accept zero or negative values that crash the loop far from the command line. Third, a video that cannot be opened during dataset extraction produces only a printed warning and two empty lists, so a class folder whose clips are all unreadable would silently contribute nothing to the training set:

```

cap = cv2.VideoCapture(str(video_path))
if not cap.isOpened():
    print(f"[Warning] Could not open video file: {video_path}")
    return [], []

```

Source: extract_dataset.py, lines 50–53 – the failure does not propagate to the caller.

Fourth, the annotated-video writer is never verified: neither `inference.py` (lines 186–188) nor `clipper.py` (line 414) calls `writer.isOpened()`, so an unsupported codec produces an empty or missing output file with no diagnostic. Fifth, the checkpoint loader in `inference.py` rebuilds the network from permissive fallbacks, including a default of two classes (line 48) that contradicts the five-class configuration; a legacy checkpoint would therefore load into a structurally wrong model instead of failing loudly:

```
checkpoint = torch.load(weights_path, map_location=device)
model = ExamBehaviorLSTM(
    input_dim=checkpoint.get("input_dim", 34),
    hidden_dim=checkpoint.get("hidden_dim", 64),
    num_layers=checkpoint.get("num_layers", 2),
    num_classes=checkpoint.get("num_classes", 2),
    bidirectional=checkpoint.get("bidirectional", True),
).to(device)
```

Source: *inference.py*, lines 43–50 – the `num_classes` fallback should be `NUM_CLASSES (5)` from *config.py*.

Sixth, the silent weight fallback in *config.py* (line 33) is itself a defensive-programming gap in disguise: suppressing the deviation means a group member may believe the documented YOLO26s-pose weights are active when the generic fallback is actually running.

Finding 4: Error and Exception Handling

Implemented Exception Handling

Five exception-handling mechanisms are verifiably implemented. The most important is the resource-lifecycle guarantee in the real-time engine: whatever happens inside the frame loop – including a `KeyboardInterrupt` from the operator – the camera, the video writer, and the display window are always released in the `finally` block:

```
finally:
    cap.release()
    if writer is not None:
        writer.release()
        print(f"[Done] Video saved successfully to: {save_video}")
    if not no_display:
        cv2.destroyAllWindows()
```

Source: *inference.py*, lines 278–284.

The clipper module demonstrates correct exception taxonomy – specific exception types with distinct handlers and defined exit codes – in its configuration loader:

```
def load_config(config_path: str) -> Dict[str, Any]:
    """Loads the clipper config JSON, exiting on error."""
    try:
        with open(config_path, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        logger.error(f"Configuration file not found at: {config_path}")
        sys.exit(1)
    except json.JSONDecodeError:
        logger.error(f"Error decoding JSON from {config_path}.")
        sys.exit(1)
```

Source: *clipper.py*, lines 436–46 (second handler message elided for space).

Batch clip extraction is isolated per clip: each export runs inside its own try block, so one corrupted segment cannot abort the remaining queue items, and failures are both logged and counted for the summary dialog (*clipper.py*, lines 413–427). Malformed configuration keys are caught as `KeyError` at GUI construction and surfaced through a modal error dialog before the window closes gracefully (*clipper.py*, lines 92–99). The dataset loader raises a deliberate, typed exception instead of returning a sentinel: `raise FileNotFoundError(f"Processed dataset not found at: {filepath}")` (*dataset.py*, lines 187–188). Finally, several modules practise

exception prevention – for instance the FPS fallbacks `cap.get(cv2.CAP_PROP_FPS)` or `30.0` (`extract_dataset.py`, line 56) and the guarded division in the live FPS calculation (`inference.py`, line 210) – which avoids raising exceptions that would then need handling.

Missing Exception Handling (Gaps)

Five gaps are identified. First, the two model-loading sites have no protection at all: `YOLO(weights_path)` in `tracker.py` (line 60) and `torch.load(weights_path, map_location=device)` in `inference.py` (line 43) will surface raw tracebacks for a corrupt or truncated weight file, with no guidance toward re-downloading or retraining. Second, the training pipeline contains no exception handling whatsoever: an interrupted run (for example, a laptop that loses power mid-epoch) abandons its history without saving partial state, and the final checkpoint reload at `train.py` line 317 is equally unguarded. Third, the broad handler in the clipper write path, `except Exception as e` (line 425), converts every possible error – including genuine programming mistakes such as a `TypeError` – into a benign “failed to save clip” log entry, which masks bugs; the narrower `cv2.error` would be more appropriate. Fourth, within that same block the writer is released only on the success path (line 422); if an exception occurs mid-write, `writer.release()` is skipped because it is not inside a `finally` clause, leaking the handle. Fifth, the extraction loop in `extract_dataset.py` has no exception handling at all, so a single corrupt frame batch aborts the entire dataset build with a traceback after minutes of processing.

Consolidated Findings and Recommendations

Table 4 merges the individual gaps from Findings 1 through 4 into a single register. Several gaps manifest in more than one category (for example, the silent video-open failure is simultaneously a defensive-programming and an exception-handling weakness); such items are listed once with their categories combined. Risk reflects the likely consequence for a group member running the project unmodified.

Table 4: Consolidated gap register with suggested remediation

Identified Gap	Category	Risk	Suggested Remediation
assert-based runtime validation (<code>dataset.py</code> 89, 123)	Defensive	Medium	Replace with explicit if / raise <code>ValueError</code>
No <code>argparse</code> range validation	Defensive	Medium	Validate threshold, epochs, stride, batch-size after <code>parse_args()</code>
Silent video-open failure in extraction (<code>extract_dataset.py</code> 50–53)	Defensive / Exceptions	High	Raise an exception or collect failures and report totals at the end
<code>VideoWriter</code> never verified (<code>inference.py</code> 186–188; <code>clipper.py</code> 414)	Defensive	Low	Check <code>writer.isOpened()</code> before the write loop
Duplicated legacy defaults, <code>num_classes=2</code> (<code>inference.py</code> 44–50)	Standards	Low	Import <code>NUM_CLASSES</code> and dimensions from <code>config.py</code>
Silent YOLO weight fallback (<code>config.py</code> 33)	Defensive / Portability	Medium	Emit a warning whenever the resolver deviates from the documented weights

Identified Gap	Category	Risk	Suggested Remediation
Unhandled YOLO / torch.load failures (tracker.py 60; inference.py 43)	Exceptions	Medium	Wrap in try/except with an actionable message
No exception handling in the training loop (train.py)	Exceptions	Medium	Guard the loop and preserve partial history on interruption
Broad except Exception; writer leak (clipper.py 425)	Exceptions	Low	Catch cv2.error; move writer.release() into finally
np.load(allow_pickle=True) (dataset.py 188)	Security	Low	Save plain arrays; document the trust boundary
print versus logging divergence	Standards / Consistency	Low	Migrate operational output to the logging module
Stale two-class docstrings (model.py 18–19; dataset.py 87)	Standards	Low	Update wording to the five-class design
No linter, formatter, or CI configuration	Standards	Low	Add a ruff or flake8 config and a minimal GitHub Actions workflow

The remediations are intentionally small. The four highest-priority items – replacing the asserts, adding CLI range validation, propagating extraction failures, and guarding model loading – together require roughly one hour of work and can be applied by any group member before the next commit cycle. The remaining items should be folded into routine maintenance before the final release; the linter and CI item, while lowest priority, would permanently prevent the standards regressions described in Finding 1 by making the verification suite run automatically on every push.

Conclusion and Overall Verdict

Measured against the four prescribed dimensions, the Automated Examination Behavior Proctoring System demonstrates deliberate and above-average attention to software quality for a prototype of this scale. Eleven professional practices are consistently applied, twelve of the fifteen quality principles are fully satisfied with three only partially satisfied, nine defensive techniques protect the perception and data boundaries, and the real-time engine plus the GUI exhibit a structured exception-handling lifecycle with guaranteed resource cleanup. The evidence is specific and reproducible: every verdict in this report resolves to a file, a line range, and an observable code construct.

The identified weaknesses form a coherent pattern rather than a random scatter: they concentrate in the batch-oriented entry points (dataset extraction and training), where silent failure, assert-based validation, and absent exception guards remain, while the interactive paths that developers exercise daily are markedly better protected. All thirteen consolidated gaps are local, low-effort fixes with no architectural impact, and the four high-priority remediations can be completed within a single short session. Once the bundled synthetic-dataset caveat noted in Section 2 is also addressed by retraining on real footage, the repository will be both measurably robust and honestly evaluated, and every group member can run, verify, and extend it by following the instructions in Section 3.